ICS Lab 2：Bomb Lab 实验报告

学号：24300240013
个人仓库：https://github.com/24300240013-fudan-edu-cn/ics-2026-bomblab

一、实验目标与方法
本实验通过分析个人 bomb 的汇编代码，恢复各关根据 challenge 计算口令的规则，并在 defuser.c 中实现求解函数。
实验的重点不是找出某个固定 seed 的答案，而是恢复适用于不同 challenge 的计算过程。使用 GDB 查看函数反汇编、跳转表、函数指针表和链表节点数据，再通过编译与运行验证实现。
主要分析流程：
1. 查看 phase_01 至 phase_06 的输入检查与函数调用。
2. 分析辅助函数中的算术运算、分支、递归和指针访问。
3. 在 defuser.c 中实现对应计算，不修改检查程序。
4. 使用公开测试和公开列表以外的 seed 验证。
5. 分析 Honor 的入口检查和附加口令计算。
符号约定：
C 表示当前关卡的 challenge。
U32(x) 表示取 x 的低 32 位。
XOR 表示按位异或，OR 表示按位或，& 表示按位与。
>> 表示无符号右移，<< 表示左移。
除特别说明外，口令计算中的加法、乘法和左移结果均按 32 位无符号整数处理，即保留低 32 位。
对 challenge 的移位先按 64 位无符号整数执行，再根据需要取低 32 位。
二、第 1 关：函数调用与参数传递
2.1 关键汇编与分析
phase_01 要求输入一个无符号整数，并将其与 p1_compute 的返回值比较：
```
mov %r12,%rdi
call p1_compute
cmp %eax,-0x14(%rbp)
sete %al
```
p1_compute 将 challenge 拆分后，依次调用 p1_step1、p1_step0 和 p1_step2。
p1_step1 的关键指令：
```
lea (%rdi,%rsi,2),%eax
xor $0xbbc4,%eax
```
p1_step0 的关键指令：
```
lea (%rdi,%rdi,2),%eax
add $0xf95f,%esi
xor %esi,%eax
```
p1_step2 的关键指令：

xor $0xe6a7,%esi
lea (%rsi,%rdi,4),%eax
LEA 在这些位置用于算术计算，而不是读取内存。例如 lea (%rdi,%rdi,2),%eax 表示计算输入的 3 倍并保留低 32 位。
参数分析不能只依赖调用约定，还需要查看被调用函数实际修改了哪些寄存器。p1_step1 没有修改 edi 和 edx，因此返回后调用者仍能分别从这些寄存器取得 hi 和 lo。
2.2 恢复出的计算关系
lo = U32(C)
hi = U32(C >> 32)
mid = U32(C >> 16)
step1 = (hi + 2 * mid) XOR 0xbbc4
step0 = (3 * lo) XOR (hi + 0xf95f)
answer = 4 * step0 + (step1 XOR 0xe6a7)
solve_phase_1 将 answer 写入 values[0]，输出一个整数。
2.3 验证结果
第一关通过全部 10 个公开 seed，并使用非公开 seed 43 验证通过。



三、第 2 关：数组、循环与递推关系
3.1 关键汇编与分析
phase_02 要求输入六个无符号整数。第一个整数的检查为：
shr $1,%rax
and $0x1f,%eax
add $0xe,%eax
cmp %eax,%esi

因此，第一个数取决于 challenge 右移一位后的低 5 位，再加 14。
后续整数通过循环检查：
lea 0xc(%rdx),%ecx
add %esi,%esi
shr %cl,%rax
and $0x1f,%eax
xor %esi,%eax
mov -0x40(%rbp,%rdx,4),%esi
add %edx,%eax
cmp %eax,%esi
rdx 为数组下标 i，从 1 递增到 5。访问地址中的比例因子 4 表明每个数组元素占 4 字节。
每轮先将前一个整数乘 2，再与 challenge 指定位置的 5 位数据异或，最后加上下标 i。
3.2 恢复出的计算关系
v[0] = ((C >> 1) & 31) + 14
对 i = 1, 2, 3, 4, 5：
bits = (C >> (i + 12)) & 31
v[i] = (bits XOR (2 * v[i-1])) + i
solve_phase_2 使用循环生成六个整数，并按顺序输出。
3.3 验证结果
第二关通过全部 10 个公开 seed，并使用非公开 seed 43 验证通过。



四、第 3 关：switch 分支与跳转表
4.1 关键汇编与分析
第三关要求输入两个无符号整数。第一个整数必须等于：

index = C & 7
第二个整数由 p3_dispatch 计算，其第二个参数为：
x = U32(C >> 3) XOR U32(C >> 32)
p3_dispatch 通过跳转表选择分支：
mov %edi,%eax
mov %esi,%edi
cmp $0x7,%eax
ja ...
notrack jmp *0x405820(,%rax,8)
这里 eax 保存表索引，edi 改为各分支函数的输入 x。每个表项占 8 字节。
使用以下命令读取实际跳转表：
x/8gx 0x405820
将表项地址与 p3_dispatch 中对应的跳转指令逐一对应，得到：
索引 0：跳转到 0x402268，再跳转到 p3_case_6。
索引 1：跳转到 0x402270，再跳转到 p3_case_3。
索引 2：跳转到 0x402278，再跳转到 p3_case_2。
索引 3：跳转到 0x402280，再跳转到 p3_case_5。
索引 4：跳转到 0x402288，再跳转到 p3_case_0。
索引 5：跳转到 0x402290，再跳转到 p3_case_1。
索引 6：跳转到 0x402258，再跳转到 p3_case_4。
索引 7：跳转到 0x402260，再跳转到 p3_case_7。
4.2 恢复出的计算关系
第一个整数为 index，第二个整数按以下规则计算：
index = 0：3 * x + 0x5c2a
index = 1：(x XOR 0x39c1) + (x >> 2)
index = 2：5 * x + 0x2c1c
index = 3：5 * x + 0x1883
index = 4：5 * x + 0xe8c6
index = 5：(2 * x) XOR 0x81d7
index = 6：(4 * x) XOR 0xf462
index = 7：(x XOR 0x4f24) + (x >> 1)
4.3 调试过程
最初实现错误地把 index 直接对应到同编号的 p3_case 函数，导致 seed 42 验证失败。seed 42 的 challenge 为 0x581ce1ff0e4ae394，其索引为 4。实际跳转表中，索引 4 对应 p3_case_0，而不是 p3_case_4。读取并核对跳转表后，重新排列全部分支，测试通过。
这一问题说明，不能依据函数名编号或函数地址顺序推断跳转表映射，必须读取实际表项。
4.4 验证结果
修正后的第三关通过全部 10 个公开 seed，并使用非公开 seed 43 验证通过。



五、第 4 关：递归调用与返回值计算

5.1 关键汇编与分析

第四关要求输入一个无符号整数，其值为 p4_walk 的返回值。初始参数为：

target = C & 127

low = 0

high = 127

salt = U32(C >> 7) XOR U32(C >> 32)

函数首先计算中点：

mov %edi,%eax

mov %edx,%edi

sub %esi,%edi

shr $1,%edi

add %esi,%edi

其中 eax 暂存 target，edi 被改为：

mid = low + (high - low) / 2

这里的除法为整数除法。

如果 target 等于 mid，执行：

lea 0x66(%rdi),%eax

xor %ecx,%eax

否则根据 target 与 mid 的大小关系递归进入左半区间或右半区间。

递归调用前，将当前层的 mid + salt 保存到 ebx。递归返回后，将子调用结果乘 3，再与该值异或，最后异或分支常数。

5.2 恢复出的递归关系

令 W(target, low, high, salt) 表示递归函数：

mid = low + (high - low) / 2

若 target == mid：
返回 (mid + 0x66) XOR salt。
若 target < mid：
child = W(target, low, mid - 1, salt)
返回 (3 * child) XOR (mid + salt) XOR 1。
若 target > mid：
child = W(target, mid + 1, high, salt)
返回 (3 * child) XOR (mid + salt) XOR 2。
target 始终在初始区间 [0,127] 中，每次递归缩小包含 target 的区间，最终命中基准情况。
实现时在 defuser.c 中添加局部辅助函数 p4_walk_local，再由 solve_phase_4 调用。
5.3 验证结果
第四关通过全部 10 个公开 seed，并使用非公开 seed 43 验证通过。



六、第 5 关：结构体布局与链表遍历
6.1 关键汇编与分析
第五关要求输入两个无符号整数。第一个是 p5_traverse 的返回值，第二个是该函数通过指针参数写出的值。
从以下访问可恢复节点布局：
mov (%rdx),%rdx
mov 0x8(%rdx),%esi
add 0xc(%rdx),%eax
节点偏移 +0：next 指针，占 8 字节。
节点偏移 +8：tag，占 4 字节。

节点偏移 +12：value，占 4 字节。
每个节点占 16 字节。
使用 GDB 读取起点表和节点数据：
x/8gx 0x405b60
x/40wx 0x405ba0
x/8wx 0x405c40
以 0x405ba0 为节点 0，按地址每增加 16 字节编号加 1，得到：
节点 0：next = 2，tag = 0x061f，value = 0x26e5。
节点 1：next = 11，tag = 0x051a，value = 0x6f93。
节点 2：next = 1，tag = 0x0842，value = 0x70ac。
节点 3：next = 0，tag = 0x011f，value = 0xb5f0。
节点 4：next = 5，tag = 0x0b15，value = 0xdba0。
节点 5：next = NULL，tag = 0x0010，value = 0xc0ef。
节点 6：next = 9，tag = 0x0206，value = 0x29a7。
节点 7：next = 3，tag = 0x0a15，value = 0x70cc。
节点 8：next = 7，tag = 0x031d，value = 0x64a0。
节点 9：next = 8，tag = 0x044d，value = 0x58b5。
节点 10：next = 4，tag = 0x092a，value = 0x1d86。
节点 11：next = 10，tag = 0x070c，value = 0xac2e。
实际链路为：
6 → 9 → 8 → 7 → 3 → 0 → 2 → 1 → 11 → 10 → 4 → 5 → NULL
起点表按索引 0 ~ 7 对应：
heads = [6, 9, 8, 7, 3, 0, 2, 1]
6.2 恢复出的计算关系
node = heads[C & 7]
steps = (C >> 3) & 3
acc = U32((C >> 6) XOR (C >> 32)) XOR 0xacb8
先处理起点，再沿 next 指针前进 steps 次。每处理一个节点执行：
acc = (3 * acc + node.value) XOR node.tag
最终输出：
第一个整数 = acc。
第二个整数 = 最后处理节点的 tag。
steps 表示前进次数，而不是处理节点数量，因此总共处理 steps + 1 个节点，即 1 ~ 4 个。
实现时使用数组保存从个人 bomb 读取的固定节点数据和 next 关系，再根据 challenge 动态选择起点和遍历次数。这些数组记录的是程序固定数据，不是某次 seed 的固定答案。
6.3 验证结果
第五关通过全部 10 个公开 seed，并使用非公开 seed 43 验证通过。

```
===== seed 43, phase 5 =====
BOMBLAB 2026                                        这里还没有明天

                              *
                               \
                            .-""""-.
                          .'        '.
                         /    LOOP    \
                        |              |
                        \              /
                         '------------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 当前] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 5 关 · 巡检回廊
这条走廊连着档案馆和街区。墙上的设备接编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0x18e47d0995aecbfe
Input (unsigned decimal integers):
Phase 5 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 已解除] [6 待解除]
```

七、第 6 关：函数指针表与间接调用

7.1 关键汇编与分析

第六关要求输入一个无符号整数，其值为 p6_dispatch 的返回值。

关键调用过程为：

and $0x38,%ebx

and $0x7,%eax

call *0x405e00(,%rax,8)

xor $0xc5b3,%eax

mov %eax,%edi

call *0x405e00(%rbx)

add $0x3c4,%eax

第一次调用使用低 3 位作为表索引。第二次调用使用 C & 0x38 作为字节偏移，因为每个表项占 8 字节，其索引为 (C >> 3) & 7。

使用以下命令读取函数表：

x/8ga 0x405e00

得到实际映射：

索引 0 → p6_op_4

索引 1 → p6_op_5

索引 2 → p6_op_1

索引 3 → p6_op_2

索引 4 → p6_op_3

索引 5 → p6_op_4

索引 6 → p6_op_5
索引 7 → p6_op_0
7.2 各函数计算规则
op0(x) = (x + 0x4f65) XOR (x >> 7)
op1(x) = ROL32(x, 2) XOR 0xecdc
op2(x) = ROR32(x, 7)
op3(x) = (4 * x + 0xe13f) XOR (3 * x)
op4(x) = ((x >> 1) XOR x) + 0xcd6e
op5(x) = (x XOR 0x7182) + (x << 7)
循环移位与普通移位不同，移出的位会回到另一端：
ROL32(x, 2) = U32(x << 2) OR (x >> 30)
ROR32(x, 7) = (x >> 7) OR U32(x << 25)
7.3 完整计算关系
令 F 表示实际函数指针表：
x = U32(C >> 6) XOR U32(C >> 32)
i = C & 7
j = (C >> 3) & 7
r1 = F[i](x)
r2 = F[j](r1 XOR 0xc5b3)
answer = r2 + 0x3c4
实现时使用辅助函数 p6_apply_local 按实际表索引选择计算规则，再完成两次调用。
7.4 验证结果
第六关通过全部 10 个公开 seed，并使用非公开 seed 43 验证通过。



八、Honor：入口检查与附加口令

8.1 输入结构分析

secret_gate 首先检查输入字符串长度：

call strlen

sub $0x24,%rax

cmp $0x7f,%rax

jbe ...

这是无符号比较，对应：

36 ≤ strlen(input) ≤ 163

随后调用 secret_copy。该函数逐字节复制输入，直到复制完结尾的空字符，不执行编码转换。

目标缓冲区从 rbp-0xd0 开始，程序检查：

cmpl $0x4e45504f,-0xb0(%rbp)

检查位置相对缓冲区的偏移为：

0xd0 - 0xb0 = 32

常数 0x4e45504f 按 x86-64 小端字节序存放，对应字符串 OPEN。

之后从偏移 36 开始调用 secret_phase：

lea -0xac(%rbp),%rdi

call secret_phase

因此构造的字符串为：

32 个 A + OPEN + 三个空格分隔的无符号整数

OPEN 后直接接第一个整数，因为 secret_phase 从偏移 36 开始读取，正好跳过前面的填充和标记。

8.2 三个整数的计算

令 K 表示 solve_secret 接收的 context。该值由驱动根据前六关的答案文本生成，不需要在求解函数中自行重算。

定义：

A = U32(C) XOR U32(C >> 32)

B = U32(K) XOR U32(K >> 32)

secret_mix 的关键指令为：

lea 0x0(,%rdi,8),%eax

sub %edi,%eax

add $0xea2a,%eax

xor $0x2694,%eax

即：

secret_mix(x) = (7 * x + 0xea2a) XOR 0x2694

该函数只修改 eax，未修改调用者随后继续使用的 esi、r8d、ecx 和 r9d。

三个整数为：

v0 = K & 3

mix = (7 * (A XOR B) + 0xea2a) XOR 0x2694

v1 = mix XOR (v0 * 0xea2a) XOR U32(K >> 16)

v2 = v1 XOR (B + v0 + 0x2694 + (A >> 5))

8.3 实现与缓冲区检查

solve_secret 使用 snprintf 将完整字符串写入 out。通过检查返回值是否为负数，以及是否达到或超过 cap，判断格式化是否失败或被截断；成功时返回 0。

输出不包含换行，换行由驱动的 puts 添加。

8.4 验证方式

查看 defuser_main.c 后确认：

./defuser --seed 43 输出前六行答案。

./defuser --secret --seed 43 只输出第七行。

Secret 模式会在内部计算前六关答案用于生成 context，但不会输出它们。

因此完整验证命令为：

```
{
    ./defuser --seed 43
    ./defuser --secret --seed 43
} | ./bomb --seed 43
```

Honor 通过全部 10 个公开 seed，并在非公开 seed 43 的完整运行中通过。

```
[SAFE] 六项检查全部通过。新存档已启用。

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。

阅读器亮起来。最先出现的不是代码，是两份很早的留言。

  许遥：

  北方那边的通知到了，我准备去。
  那个项目我想试很久了。你帮我看申请的时候，我还说不一定选得上，
  现在真能去了，反而有点不习惯。以后遇到问题，
  不能再把椅子滑到你旁边，问一句‘你看这是不是有毛病’。

  你想留下继续手里的事，我记得你为它准备了多久。
  我也想看看，最后能做成什么样。
  这里没做完的几处，我把记录整理好了。
  你有空就改，没空先忙自己的事。

  走之前出去吃顿饭吧。这次别在实验室凑合了。

  —— sky
```

九、测试汇总

选用不在公开列表中的 seed 43，对前六关分别验证，并进行包括 Honor 在内的完整验证。

第 1 关：公开测试 10/10，非公开 seed 43 通过。

第 2 关：公开测试 10/10，非公开 seed 43 通过。

第 3 关：公开测试 10/10，非公开 seed 43 通过。

第 4 关：公开测试 10/10，非公开 seed 43 通过。

第 5 关：公开测试 10/10，非公开 seed 43 通过。

第 6 关：公开测试 10/10，非公开 seed 43 通过。

Honor：公开测试 10/10，非公开 seed 43 通过。

```
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

十、实验总结

本实验将汇编中的寄存器操作、内存访问和控制流恢复为 C 语言计算过程，主要涉及：

1. 根据调用约定及函数实际指令分析参数和返回值。
2. 区分 64 位 challenge 运算与 32 位口令运算。
3. 从循环恢复数组递推关系。
4. 读取真实跳转表，而不是按函数编号猜测分支映射。
5. 从递归调用参数和返回处理恢复递归公式。
6. 根据字段偏移识别结构体布局，并沿实际指针恢复链表。
7. 区分函数指针表索引与字节偏移。
8. 正确实现循环移位、小端字节序及字符串长度检查。

调试中最明显的问题出现在第三关：最初误将跳转表索引等同于函数编号。通过读取跳转表并逐项对应目标指令，修正分支映射后通过测试。这说明反汇编分析必须依据实际控制流和数据，不能仅凭命名推断。

十一、参考资料与辅助工具使用说明

11.1 参考资料

1. 课程 Bomb Lab 实验页面：https://ics-26fall-fdu.github.io/labs/lab2-bomb-lab/
2. BombLab 学生模板仓库：https://github.com/ICS-26Fall-FDU/BombLab
3. 个人仓库中的 README.md、defuser.c 模板、defuser_main.c 及命令行帮助信息。

11.2 辅助工具及用途

GDB：查看反汇编、跳转表、函数指针表、链表节点和内存布局。

GCC / make：编译求解器，检查编译警告与错误。

public_test.py：执行公开测试。

Git：管理求解代码和实验文件。

ChatGPT：协助解释汇编、分析测试失败原因，整理报告草稿。